

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force\_remount=True)

## Load data

```
import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/CPE393/data.csv')
df.head()
```

|   | Make | Model    | Year | Engine Fuel Type            | Engine HP | Engine Cylinders | Transmission Type | Driven_Wheels    | Number of Doors | Market Category                       | Vehicle Size | Vehicle Style |
|---|------|----------|------|-----------------------------|-----------|------------------|-------------------|------------------|-----------------|---------------------------------------|--------------|---------------|
| 0 | BMW  | Series M | 2011 | premium unleaded (required) | 335.0     | 6.0              | MANUAL            | rear wheel drive | 2.0             | Factory Tuner,Luxury,High-Performance | Compact      | Coupe         |
| 1 | BMW  | Series   | 2011 | premium unleaded (required) | 300.0     | 6.0              | MANUAL            | rear wheel drive | 2.0             | Luxury,Performance                    | Compact      | Convertible   |
| 2 | BMW  | Series   | 2011 | premium unleaded (required) | 300.0     | 6.0              | MANUAL            | rear wheel drive | 2.0             | Luxury,High-Performance               | Compact      | Coupe         |
|   |      | 1        |      | premium                     |           |                  |                   |                  |                 |                                       |              |               |

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df['Maker_Model'] = df['Make'] + "-" + df['Model']
```

df

|       | Make  | Model    | Year | Engine Fuel Type            | Engine HP | Engine Cylinders | Transmission Type | Driven_Wheels    | Number of Doors | Market Category                       | Vehicle Size | Vehicle Style |
|-------|-------|----------|------|-----------------------------|-----------|------------------|-------------------|------------------|-----------------|---------------------------------------|--------------|---------------|
| 0     | BMW   | Series M | 2011 | premium unleaded (required) | 335.0     | 6.0              | MANUAL            | rear wheel drive | 2.0             | Factory Tuner,Luxury,High-Performance | Compact      | Coupe         |
| 1     | BMW   | Series   | 2011 | premium unleaded (required) | 300.0     | 6.0              | MANUAL            | rear wheel drive | 2.0             | Luxury,Performance                    | Compact      | Convertible   |
| 2     | BMW   | Series   | 2011 | premium unleaded (required) | 300.0     | 6.0              | MANUAL            | rear wheel drive | 2.0             | Luxury,High-Performance               | Compact      | Coupe         |
| 3     | BMW   | Series   | 2011 | premium unleaded (required) | 230.0     | 6.0              | MANUAL            | rear wheel drive | 2.0             | Luxury,Performance                    | Compact      | Convertible   |
| 4     | BMW   | Series   | 2011 | premium unleaded (required) | 230.0     | 6.0              | MANUAL            | rear wheel drive | 2.0             | Luxury                                | Compact      | Coupe         |
| ...   | ...   | ...      | ...  | ...                         | ...       | ...              | ...               | ...              | ...             | ...                                   | ...          | ...           |
| 11909 | Acura | ZDX      | 2012 | premium unleaded (required) | 300.0     | 6.0              | AUTOMATIC         | all wheel drive  | 4.0             | Crossover,Hatchback,Luxury            | Compact      | Coupe         |
| 11910 | Acura | ZDX      | 2012 | premium unleaded (required) | 300.0     | 6.0              | AUTOMATIC         | all wheel drive  | 4.0             | Crossover,Hatchback,Luxury            | Compact      | Coupe         |
| 11911 | Acura | ZDX      | 2012 | premium unleaded (required) | 300.0     | 6.0              | AUTOMATIC         | all wheel drive  | 4.0             | Crossover,Hatchback,Luxury            | Compact      | Coupe         |

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Select features from original dataset to form a new dataframe
df1 = df[['Engine Fuel Type', 'Transmission Type', 'Driven_Wheels', 'Market Category', 'Vehicle Size', 'Vehicle Style', 'Maker_Model']]
df1.head()
```

|   | Engine                      | Fuel Type | Transmission Type | Driven_Wheels    | Market Category                       | Vehicle Size | Vehicle Style | Maker_Model    |
|---|-----------------------------|-----------|-------------------|------------------|---------------------------------------|--------------|---------------|----------------|
| 0 | premium unleaded (required) |           | MANUAL            | rear wheel drive | Factory Tuner,Luxury,High-Performance | Compact      | Coupe         | BMW 1 Series M |
| 1 | premium unleaded (required) |           | MANUAL            | rear wheel drive | Luxury,Performance                    | Compact      | Convertible   | BMW 1 Series   |
| 2 | premium unleaded (required) |           | MANUAL            | rear wheel drive | Luxury,High-Performance               | Compact      | Coupe         | BMW 1 Series   |

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
# For each row, combine all the columns into one column
df2 = df1.apply(lambda x: ', '.join(x.astype(str)), axis=1)
df2.head()
```

|   | 0                                                 |
|---|---------------------------------------------------|
| 0 | premium unleaded (required),MANUAL,rear wheel ... |
| 1 | premium unleaded (required),MANUAL,rear wheel ... |
| 2 | premium unleaded (required),MANUAL,rear wheel ... |
| 3 | premium unleaded (required),MANUAL,rear wheel ... |
| 4 | premium unleaded (required),MANUAL,rear wheel ... |

dtype: object

df2.shape

(11914, )

```
# Store them in a pandas dataframe
df_clean = pd.DataFrame({'clean': df2})
df_clean.head()
```

|   | clean                                             |
|---|---------------------------------------------------|
| 0 | premium unleaded (required),MANUAL,rear wheel ... |
| 1 | premium unleaded (required),MANUAL,rear wheel ... |
| 2 | premium unleaded (required),MANUAL,rear wheel ... |
| 3 | premium unleaded (required),MANUAL,rear wheel ... |
| 4 | premium unleaded (required),MANUAL,rear wheel ... |

Next steps: [Generate code with df\\_clean](#) [New interactive sheet](#)

```
# Create the list of list format of the custom corpus for gensim modeling
sent = [row.split(',') for row in df_clean['clean']]
# show the example of list of list format of the custom corpus for gensim modeling
sent[:2]
```

```
[['premium unleaded (required)',
 'MANUAL',
 'rear wheel drive',
 'Factory Tuner',
 'Luxury',
 'High-Performance',
 'Compact',
 'Coupe',
 'BMW 1 Series M'],
 ['premium unleaded (required)',
 'MANUAL',
 'rear wheel drive',
 'Luxury',
 'Performance',
 'Compact',
 'Convertible',
 'BMW 1 Series']]
```

✓ Train Word2Vec model (skip-gram)

```
!pip install gensim
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.3.3)
Requirement already satisfied: numpy<2.0,>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.26.4)
Requirement already satisfied: scipy<1.14.0,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.13.1)
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.3.1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open>=1.8.1->gensim) (1.17.3)
```

```
# from gensim import models
from gensim.models import Word2Vec
model = Word2Vec(sent, min_count=1,vector_size= 300,workers=5, window=10, sg = 1, epochs=100)
# model = Word2Vec(sent, min_count=1,size= 300,workers=5, window=10, sg = 1, iter=100)
```

**vector\_size:** The number of dimensions of the embeddings and the default is 100.

**window:** The maximum distance between a target word and words around the target word. The default window is 5.

**min\_count:** The minimum count of words to consider when training the model; words with occurrence less than this count will be ignored. The default for min\_count is 5.

**workers:** The number of partitions during training and the default workers is 3.

**sg:** The training algorithm, either CBOW(0) or skip gram(1). The default training algorithm is CBOW.

```
model.wv['Toyota Camry']
```

```
array([ 1.47385404e-01,  6.27801269e-02,  5.33101559e-02, -9.07321200e-02,
 1.39081225e-01, -8.97382870e-02, -2.05423608e-02, -3.65444459e-02,
-8.46286938e-02,  5.02994023e-02,  1.93774566e-01, -8.53577405e-02,
 1.73119336e-01,  1.94758400e-01,  6.14983379e-04, -4.27879244e-02,
 2.30320871e-01,  8.36646929e-02, -7.95617923e-02, -3.30228843e-02,
-9.66270268e-02, -1.28960282e-01, -1.83718860e-01,  1.95413351e-01,
-1.88748121e-01, -1.03352770e-01, -1.71399713e-01,  1.53146967e-01,
-6.38812631e-02, -1.54884025e-01,  4.09680575e-01,  6.94107078e-03,
-9.48411748e-02, -5.39955720e-02,  2.32925955e-02, -1.00150250e-01,
 7.20573291e-02, -1.41351953e-01, -1.26193436e-02,  5.77153713e-02,
-3.10708404e-01,  2.95946607e-03,  2.08722055e-01,  2.49710798e-01,
-1.02214724e-01,  6.94779232e-02, -4.60041687e-02, -7.07566589e-02,
 2.12824598e-01,  2.20088363e-02,  3.17619406e-02, -3.06416631e-01,
-1.42771574e-02, -4.27323431e-02, -8.33935067e-02,  4.69498485e-02,
-4.21317592e-02, -1.75405443e-02, -1.18653283e-01,  3.79733890e-02,
 1.30541712e-01,  1.53483436e-01,  1.38790086e-01,  3.29342335e-01,
-2.44266421e-01,  1.07755378e-01,  1.12170741e-01,  4.58819699e-03,
-1.33785084e-01,  3.56825113e-01,  2.57515520e-01,  2.70005781e-03,
 2.78773963e-01, -1.53028831e-01,  1.87086940e-01,  1.76580608e-01,
-1.05168700e-01,  2.91968789e-02, -2.73924880e-02, -2.46761134e-03,
 3.41359340e-02, -7.24703595e-02, -8.66024718e-02,  3.70710135e-01,
 2.22945616e-01, -5.38265370e-02,  3.36515158e-01,  5.36741987e-02,
 1.64970998e-02,  3.65917291e-03,  2.06042364e-01, -5.11449985e-02,
-3.58290315e-01, -1.02528401e-01, -6.24210835e-02, -1.40172645e-01,
 1.69544756e-01, -2.29905900e-02,  6.15031384e-02, -1.44965023e-01,
 2.55984455e-01,  1.78526431e-01, -5.99167962e-03,  6.06335923e-02,
 1.15501940e-01,  3.30477543e-02,  1.89171478e-01, -1.34606138e-01,
-3.72652203e-01, -1.21175405e-02, -1.67862698e-01,  3.08949322e-01,
-1.01074100e-01,  1.13883190e-01,  7.62372091e-02,  9.99588892e-02,
 3.62806350e-01, -3.14574718e-05,  1.66465342e-02, -2.32398853e-01,
-1.20449178e-01,  3.51775497e-01,  1.31362110e-01, -7.52316564e-02,
 2.02742815e-01,  3.40537339e-01, -4.44815516e-01, -1.35658860e-01,
-1.11911237e-01,  5.86803436e-01,  1.93836197e-01, -3.85497548e-02,
 1.56719834e-01, -7.35490859e-01,  1.00205839e-01,  5.54168761e-01,
-1.38941512e-01, -1.73218951e-01, -5.31407237e-01, -3.00764620e-01,
 1.12900265e-01,  6.94175214e-02, -1.74280614e-01,  1.86251104e-01,
 3.70776430e-02, -6.10848516e-02, -7.53861889e-02, -1.48964867e-01,
 1.67954743e-01, -3.06506306e-01,  2.56459475e-01, -2.43345991e-01,
-1.45388573e-01, -1.44298365e-02, -2.54542679e-01,  1.10272802e-01,
-1.96847796e-01,  3.62911701e-01, -1.98536992e-01,  3.55805367e-01,
-1.02689918e-02, -4.44925949e-02, -4.58968617e-02,  3.16560358e-01,
 4.90127176e-01, -1.79573968e-02, -1.63741037e-01, -1.34554148e-01,
-1.06768698e-01,  7.52804428e-02, -3.40933084e-01, -9.53248143e-02,
 5.10482527e-02,  2.65351888e-02,  9.89441350e-02, -1.03235871e-01,
-3.02563727e-01,  1.16483368e-01,  1.81047961e-01, -6.62247166e-02,
 5.10438457e-02, -3.03000608e-03, -1.80900931e-01, -1.37660831e-01,
-2.99591959e-01, -1.20989002e-01,  3.76232296e-01,  2.62131840e-01,
 2.17937335e-01, -1.69051409e-01,  1.98477656e-01,  1.90866277e-01,
-1.66146234e-01, -4.66466211e-02, -1.14338644e-01, -3.28314751e-01,
-9.99808908e-02,  1.66004881e-01, -3.65455411e-02, -1.32176206e-01,
-1.63864315e-01,  1.60449415e-01, -1.36866525e-01, -7.56140351e-02,
 2.41085485e-01,  3.13935518e-01,  1.53747305e-01,  1.14685170e-01,
-1.26453862e-01,  1.57965392e-01, -9.70207900e-02, -1.47748023e-01,
 2.93322891e-01, -2.01730970e-02,  1.10500880e-01,  1.70654915e-02,
-6.51441514e-02, -4.55606878e-01, -2.04373494e-01, -2.48579383e-01,
 5.84581196e-02, -1.69774562e-01,  1.97300240e-01, -2.56103456e-01,
-7.41054490e-02,  7.31063774e-03,  1.99231207e-01, -4.67638746e-02,
 7.09641352e-02, -9.69673544e-02,  5.29753752e-02, -1.69667169e-01,
```

```
model.wv.similarity('Porsche 718 Cayman', 'Nissan Van')
```

```
0.54736257
```

```
model.wv.similarity('Porsche 718 Cayman', 'Mercedes-Benz SLK-Class')
```

```
0.8255832
```

```
model.wv.most_similar('Mercedes-Benz SLK-Class')[:5]
```

```
[('Mercedes-Benz SL-Class', 0.9842197299003601),
 ('Cadillac XLR-V', 0.9775272607803345),
 ('Mercedes-Benz SLC-Class', 0.9453487992286682),
 ('BMW Z4 M', 0.9263644218444824),
 ('Porsche Boxster', 0.9226880073547363)]
```

```
model
```

```
<gensim.models.word2vec.Word2Vec at 0x7ab300241460>
```

```
import numpy as np
def cosine_distance (model, word,target_list , num) :
    cosine_dict ={}
    word_list = []
    a = model.wv[word]
    for item in target_list :
        if item != word :
            b = model.wv[item]
            cos_sim = np.dot(a, b)/(np.linalg.norm(a)*np.linalg.norm(b))
            cosine_dict[item] = cos_sim
    dist_sort=sorted(cosine_dict.items(), key=lambda dist: dist[1],reverse = True) ## in Descending order
    for item in dist_sort:
        word_list.append((item[0], item[1]))
    return word_list[0:num]
```

```
# only get the unique Maker_Model
Maker_Model = list(df.Maker_Model.unique())

# Show the most similar Mercedes-Benz SLK-Class by cosine distance
cosine_distance(model,'Mercedes-Benz SLK-Class',Maker_Model,5)
```

```
[('Mercedes-Benz SL-Class', 0.98421973),
 ('Cadillac XLR-V', 0.9775273),
 ('Mercedes-Benz SLC-Class', 0.9453488),
 ('BMW Z4 M', 0.9263644),
 ('Porsche Boxster', 0.922688)]
```

## Activity

1. Use the customer complaint data to estimate the word embedding vectors
2. Find similar words to 'debt', 'collection', 'risk'

```
import pandas as pd
df = pd.read_pickle('/content/drive/MyDrive/CPE393/consumer_complaint_dataset.data', compression='gzip')
```

## EDA

```
df.head(10)
```

|   | topic                                             | input                                             |  |
|---|---------------------------------------------------|---------------------------------------------------|--|
| 0 | Debt collection                                   | transworld systems inc. \nis trying to collect... |  |
| 1 | Credit reporting, credit repair services, or o... | I would like to request the suppression of the... |  |
| 2 | Debt collection                                   | Over the past 2 weeks, I have been receiving e... |  |
| 3 | Credit reporting, credit repair services, or o... | I HAD FILED WITH CFPB ON XX/XX/XXXX19 TO HAVE ... |  |
| 4 | Credit reporting, credit repair services, or o... | I have several accounts that the balance is in... |  |
| 5 | Money transfer, virtual currency, or money ser... | I was sold access to an event digitally, of wh... |  |
| 6 | Debt collection                                   | While checking my credit report I noticed thre... |  |
| 7 | Credit reporting, credit repair services, or o... | I would like the credit bureau to correct my X... |  |
| 8 | Credit reporting, credit repair services, or o... | MY NAME IS XXXX XXXX XXXX I live at XXXX XXXX ... |  |
| 9 | Credit reporting, credit repair services, or o... | Today XX/XX/XXXX went online to dispute the in... |  |

```
df.shape
```

```
(492255, 2)
```

```
df['topic'].value_counts()
```

|                                                                                     | count  |
|-------------------------------------------------------------------------------------|--------|
| <b>Credit reporting, credit repair services, or other personal consumer reports</b> | 145090 |
| <b>Debt collection</b>                                                              | 106946 |
| <b>Mortgage</b>                                                                     | 61581  |
| <b>Credit card or prepaid card</b>                                                  | 32144  |
| <b>Credit reporting</b>                                                             | 31588  |
| <b>Student loan</b>                                                                 | 25083  |
| <b>Checking or savings account</b>                                                  | 19153  |
| <b>Credit card</b>                                                                  | 18838  |
| <b>Bank account or service</b>                                                      | 14885  |
| <b>Consumer Loan</b>                                                                | 9473   |
| <b>Vehicle loan or lease</b>                                                        | 8204   |
| <b>Money transfer, virtual currency, or money service</b>                           | 7865   |
| <b>Payday loan, title loan, or personal loan</b>                                    | 6404   |
| <b>Payday loan</b>                                                                  | 1746   |
| <b>Money transfers</b>                                                              | 1497   |
| <b>Prepaid card</b>                                                                 | 1450   |
| <b>Other financial service</b>                                                      | 292    |
| <b>Virtual currency</b>                                                             | 16     |

```
dtype: int64
```

```
df['sum'] = df['topic'] + ' ' + df['input']
df['sum'].head()
```

|   | sum                                               |
|---|---------------------------------------------------|
| 0 | Debt collection transworld systems inc. \nis t... |
| 1 | Credit reporting, credit repair services, or o... |
| 2 | Debt collection Over the past 2 weeks, I have ... |
| 3 | Credit reporting, credit repair services, or o... |
| 4 | Credit reporting, credit repair services, or o... |

```
dtype: object
```

```
input_text = df['sum'].tolist()
input_text[:10]
```

- Prepare Data

```
import re
from nltk.corpus import stopwords, wordnet
from nltk.stem import WordNetLemmatizer
from nltk import pos_tag

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

# remove masked info
def remove_masking(text):
    text = text.lower()
    masked_pattern = re.compile(r'xx+', re.IGNORECASE)
    return masked_pattern.sub('', text)

# POS tag
def get_wordnet_pos(tag):
```

```

    if tag.startswith('J'):
        return wordnet.ADJ
    elif tag.startswith('V'):
        return wordnet.VERB
    elif tag.startswith('N'):
        return wordnet.NOUN
    elif tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN

# clean + tokenize + lemmatize
def clean_text(text: str):
    text = remove_masking(text)
    text = re.sub(r'^a-z\s', '', text)
    tokens = text.split()
    pos_tags = pos_tag(tokens)

    lemmatized = [
        lemmatizer.lemmatize(w, get_wordnet_pos(tag))
        for w, tag in pos_tags if w not in stop_words
    ]
    return lemmatized

# ใช้กับ list ของข้อความ
def preprocess_text(text_list):
    return [clean_text(text) for text in text_list]

```

```

sample = ["Cats are running in the fields. xxxx",
          "Children studied harder for exams."]

```

```
print(preprocess_text(sample))
```

```
[['cat', 'run', 'field'], ['child', 'study', 'harder', 'exam']]
```

```
text = preprocess_text(input_text)
```

```

with open("/content/drive/MyDrive/CPE393/preprocess_text.txt", "w", encoding="utf-8") as f:
    for doc in text:
        f.write(" ".join(doc) + "\n")

```

```

# Train Word2Vec model
w2v_model = Word2Vec(sentences=text,
                     vector_size=100, # embedding size
                     window=10,      # context window
                     min_count=1,
                     workers=5,
                     sg=1,            # skip-gram
                     epochs=10)

```

```

print(f"Vocabulary size: {len(model.wv)}")
print("Sample words:", list(model.wv.index_to_key[:20]))

```

```

Vocabulary size: 977
Sample words: ['AUTOMATIC', 'regular unleaded', 'front wheel drive', 'Compact', 'Midsize', 'nan', 'rear wheel drive', 'Luxury',

```

```
w2v_model.save("/content/drive/MyDrive/CPE393/complaints_w2v.model")
```

```

words_to_check = ["debt", "collection", "risk"]

for word in words_to_check:
    if word in w2v_model.wv.key_to_index: # ตรวจสอบว่ามีใน vocab
        similar_words = w2v_model.wv.most_similar(word, topn=10)
        print(f"similar '{word}':")
        for sim_word, score in similar_words:
            print(f"    {sim_word}: {score:.4f}")
        print("-" * 30)
    else:
        print(f"'{word}' not in vocab")
        print("-" * 30)

```

```

similar 'debt':
collection: 0.8088
collector: 0.8002
allged: 0.7636

```

```

reponddispute: 0.7626
outsourcings: 0.7623
outsourcing: 0.7595
direcmanagement: 0.7519
receiveable: 0.7470
fromamountdate: 0.7454
outsorcing: 0.7398
-----
similar 'collection':
debt: 0.8088
businesscollection: 0.7706
receiveable: 0.7606
natlcrsys: 0.7529
floridabased: 0.7493
incregarding: 0.7429
systemspursuant: 0.7333
outsourcing: 0.7324
carpio: 0.7297
originalactual: 0.7271
-----
similar 'risk':
everpresent: 0.7097
wwwequifaxsecurity: 0.7094
liferuining: 0.7053
mpact: 0.6900
exposure: 0.6893
misshandling: 0.6790
reputational: 0.6700
equifaxsecuirtycom: 0.6698
highalert: 0.6672
itcybersecurity: 0.6556
-----

```

```

import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
import numpy as np

targets = words_to_check
topn = 10

words = []
vectors = []
colors = []

color_map = {
    "debt": "#FFD93D",
    "collection": "#4ECDC4",
    "risk": "#FF6B6B"
}

for target in targets:
    if target in w2v_model.wv:
        sims = [w for w, _ in w2v_model.wv.most_similar(target, topn=topn)]
    else:
        sims = []

    # target word itself
    words.append(target)
    vectors.append(w2v_model.wv[target])
    colors.append(color_map[target])

    # similar words
    for w in sims:
        words.append(w)
        vectors.append(w2v_model.wv[w])
        colors.append(color_map[target])

vectors = np.array(vectors)

tsne = TSNE(n_components=2, random_state=42, perplexity=5)
vectors_2d = tsne.fit_transform(vectors)

plt.figure(figsize=(15,8))
for i, word in enumerate(words):
    plt.scatter(vectors_2d[i,0], vectors_2d[i,1], color=colors[i], s=50)
    plt.text(vectors_2d[i,0]+0.1, vectors_2d[i,1]+0.1, word, fontsize=9)

plt.title("t-SNE of similar words")
plt.show()

```



